

Backtracking e Força Bruta

1. Modelagem do Problema

1.1 Estrutura do Tabuleiro

O tabuleiro do jogo foi representado por uma matriz quadrada de dimensão $N \times N$, sendo N um número par. Cada posição da matriz pode assumir um de três estados: **Sol (S)**, **Lua (L)** ou **vazia (.)**, que representa uma célula ainda não preenchida durante a resolução do problema.

Constante	Valor	Símbolo	Significado
SOL	1	S	Representa o símbolo Sol
LUA	0	L	Representa o símbolo Lua
VAZIO	-1	.	Indica que a célula ainda não foi preenchida

Para representar o tabuleiro foi criada a classe `Tabuleiro`, responsável por armazenar todas as informações necessárias para a resolução do quebra-cabeça. Essa classe mantém as dimensões da matriz (linhas e colunas), o estado atual de cada célula (`matriz[][]`) e a lista de restrições existentes entre as posições do tabuleiro.

Além do armazenamento dos dados, a classe também disponibiliza métodos que facilitam a manipulação do tabuleiro durante a execução dos algoritmos, como obter e alterar valores das células (`getValor` e `setValor`), verificar se uma posição ainda está vazia (`isVazio`), criar uma cópia completa do tabuleiro (`copiar`), localizar as posições não preenchidas (`getPosicoesVazias`) e calcular a quantidade de símbolos que cada linha e coluna deve possuir (`getMetade`).

1.2 Estrutura das Restrições

As restrições entre as células foram representadas pela classe `Restricao`. Cada restrição relaciona duas posições do tabuleiro, representadas pela classe `Posicao`, e um tipo de relacionamento definido pela enumeração `TipoRestricao`.

Foram considerados dois tipos de restrições:

- **IGUAL (=)**: as duas células devem obrigatoriamente conter o mesmo símbolo, atendendo à Regra 4 do problema.
- **OPOSIÇÃO (×)**: as duas células devem conter símbolos diferentes, conforme definido na Regra 5.

Essa modelagem permitiu separar as restrições da estrutura principal do tabuleiro, tornando o código mais organizado e facilitando tanto a validação das regras quanto a implementação dos algoritmos de busca.

1.3 Representação dos Símbolos

Embora o usuário visualize os símbolos **Sol (S)** e **Lua (L)** no arquivo de entrada e na saída do programa, internamente eles são representados por valores inteiros, o que simplifica as comparações realizadas pelos algoritmos.

Para realizar essa conversão foram implementados dois métodos na classe Tabuleiro:

- `charParaValor(char)`: converte os caracteres lidos do arquivo (S, L ou .) para a representação interna utilizada pelo programa.
- `valorParaChar(int)`: realiza a operação inversa, convertendo os valores internos para caracteres antes da impressão do tabuleiro no console.

Essa abordagem permite separar a forma como os dados são armazenados da forma como são apresentados ao usuário, deixando a implementação mais organizada e facilitando futuras alterações na representação dos símbolos, caso necessário.

1.4 Estrutura das Classes

O projeto foi organizado em classes com responsabilidades bem definidas, buscando facilitar a manutenção e a reutilização do código. Cada classe é responsável por uma parte específica do sistema, evitando que diferentes funcionalidades fiquem concentradas em um único arquivo.

A organização das principais classes é apresentada a seguir:

Classe	Responsabilidade
Main	Coordena a execução do programa, realizando a leitura do arquivo, a impressão do tabuleiro e a chamada dos algoritmos de resolução.
Tabuleiro	Representa o estado atual do jogo, armazenando a matriz e as restrições existentes.
Posicao	Representa uma posição do tabuleiro por meio das coordenadas de linha e coluna.
Restricao	Armazena duas posições relacionadas e o tipo da restrição entre elas.
TipoRestricao	Enumeração responsável por definir os tipos de restrição (IGUAL e OPOSICAO).
LeitorArquivo	Realiza a leitura e o processamento dos arquivos de entrada.
Validador	Centraliza toda a lógica responsável por verificar se as regras do jogo estão sendo respeitadas.
ForcaBrutaSolver	Implementa a estratégia de busca exaustiva, testando todas as combinações possíveis.
BacktrackingSolver	Implementa a busca por retrocesso, utilizando podas para reduzir o espaço de busca.
ResultadoAlgoritmo	Armazena a solução encontrada e as estatísticas de execução de cada algoritmo.
ImpressoraTabuleiro	Responsável por exibir o tabuleiro de forma organizada no terminal.

Durante o desenvolvimento foram aplicados alguns princípios básicos de orientação a objetos. Cada classe possui uma responsabilidade específica, reduzindo o acoplamento entre os componentes e facilitando a manutenção do sistema. Além disso, toda a lógica de validação das regras foi concentrada na classe Validador, permitindo que os algoritmos de Força Bruta e Backtracking reutilizem as mesmas verificações sem duplicação de código.

2. Estratégia da Força Bruta

2.1 Geração do Espaço de Estados

A estratégia de Força Bruta consiste em explorar todas as possibilidades de preenchimento das células vazias do tabuleiro, sem realizar qualquer tipo de otimização durante a busca.

Inicialmente, o programa identifica todas as posições que ainda não foram preenchidas por meio do método `getPosicoesVazias()`. Em seguida, o algoritmo percorre essa lista de forma recursiva, atribuindo, para cada posição, um dos dois valores possíveis: **Sol (1)** ou **Lua (0)**.

Como cada célula vazia possui apenas duas possibilidades de preenchimento, um tabuleiro com **k** posições vazias gera um espaço de busca composto por 2^k combinações possíveis. Dessa forma, o algoritmo garante que todas as configurações do problema sejam analisadas.

2.2 Validação das Soluções

Diferentemente do Backtracking, a implementação da Força Bruta não realiza verificações durante o preenchimento do tabuleiro. As regras do jogo são avaliadas apenas quando todas as posições vazias já receberam um valor.

Nesse momento, é criada uma instância da classe Validador, responsável por executar o método `tabuleiroValido()`. Esse método verifica se a configuração encontrada atende às cinco regras do quebra-cabeça Tango, incluindo o equilíbrio entre Sóis e Luas, as restrições de adjacência e as restrições de igualdade e oposição.

Caso todas as regras sejam satisfeitas, a solução é considerada válida e o algoritmo é encerrado. Caso contrário, a busca continua até que todas as combinações possíveis tenham sido analisadas.

2.3 Vantagens e Desvantagens

A principal vantagem da Força Bruta é sua simplicidade. O algoritmo é relativamente fácil de implementar e garante que, caso exista uma solução, ela será encontrada, já que nenhuma possibilidade é descartada antes da verificação final.

Por outro lado, essa abordagem apresenta um custo computacional elevado. Como todas as combinações são exploradas, o número de estados cresce exponencialmente conforme aumenta a quantidade de células vazias. Além disso, muitos estados claramente inválidos continuam sendo expandidos até o final, desperdiçando tempo de processamento por não haver qualquer mecanismo de poda.

2.4 Complexidade

Considerando um tabuleiro com **k** posições vazias, o algoritmo pode gerar até 2^k combinações diferentes. Para cada configuração completa, é necessário verificar todas as regras do jogo, o que resulta em uma complexidade temporal de aproximadamente $O(2^k \times n^2)$, sendo **n** a dimensão do tabuleiro.

Em relação ao consumo de memória, a implementação utiliza apenas a pilha de chamadas recursivas e algumas estruturas auxiliares, resultando em uma complexidade espacial de $O(k)$, correspondente à profundidade máxima da recursão.

3. Estratégia do Backtracking

3.1 Funcionamento da Função Recursiva

O algoritmo de Backtracking utiliza uma abordagem recursiva para preencher o tabuleiro de forma incremental. Inicialmente, é obtida a lista de posições vazias e, a cada chamada recursiva, uma dessas posições é selecionada para receber um dos dois valores possíveis: **Sol** ou **Lua**.

Após cada atribuição, o algoritmo verifica imediatamente se o estado atual ainda pode levar a uma solução válida. Caso nenhuma regra tenha sido violada, a busca continua para a próxima posição vazia. Caso contrário, o algoritmo desfaz a última atribuição e tenta o outro valor disponível. Esse processo de retornar ao estado anterior sempre que uma tentativa falha é conhecido como *backtracking* (retrocesso).

3.2 Caso Base

A condição de parada ocorre quando todas as posições vazias já foram preenchidas. Nesse momento, o tabuleiro representa uma solução completa e é realizada uma validação final por meio do método `tabuleiroValido()`. Se todas as regras do quebra-cabeça forem satisfeitas, a solução é armazenada e o algoritmo é encerrado.

3.3 Passo Recursivo

Durante cada etapa da busca, o algoritmo executa as seguintes operações:

1. Seleciona a próxima posição vazia do tabuleiro.
2. Tenta preencher essa posição com **Sol** e, em seguida, com **Lua**.
3. Após cada tentativa, realiza uma validação parcial apenas das regras relacionadas à posição recém-preenchida.
4. Caso o estado continue válido, a busca prossegue para a próxima posição.
5. Caso alguma restrição seja violada, a tentativa é descartada, o contador de podas é incrementado e o algoritmo retorna ao estado anterior para testar outra possibilidade.

Essa estratégia evita que o algoritmo continue explorando caminhos que já são conhecidos como inválidos.

3.4 Critérios de Poda

A principal diferença entre o Backtracking e a Força Bruta está na utilização de podas durante a construção da solução. Em vez de esperar o preenchimento completo do tabuleiro para verificar as regras, o algoritmo identifica inconsistências assim que elas surgem, eliminando imediatamente esses ramos da árvore de busca.

As podas implementadas foram as seguintes:

Critério de poda	Regra aplicada	Descrição
Adjacência horizontal	Regra 2	Interrompe a busca quando três símbolos iguais aparecem consecutivamente em uma linha.
Adjacência vertical	Regra 2	Interrompe a busca quando três símbolos iguais aparecem

		consecutivamente em uma coluna.
Equilíbrio parcial das linhas	Regra 3	Impede que uma linha possua mais Sóis ou Luas do que o permitido antes mesmo de ser completada.
Equilíbrio parcial das colunas	Regra 3	Impede que uma coluna ultrapasse o limite de símbolos permitido.
Restrições de igualdade (=)	Regra 4	Descarta estados em que duas posições ligadas por = recebem símbolos diferentes.
Restrições de oposição (×)	Regra 5	Descarta estados em que duas posições ligadas por × recebem o mesmo símbolo.

Essas verificações reduzem significativamente o número de estados explorados, pois impedem que o algoritmo desperdice tempo analisando configurações que nunca poderão resultar em uma solução válida.

3.5 Vantagens

A utilização de podas torna o Backtracking muito mais eficiente do que a Força Bruta na resolução do quebra-cabeça Tango. Como estados inválidos são descartados logo no início da busca, o algoritmo percorre um número muito menor de combinações e, conseqüentemente, encontra a solução em menos tempo.

Além do ganho de desempenho, essa abordagem se adapta naturalmente ao problema, já que as restrições do jogo podem ser verificadas à medida que o tabuleiro é construído.

3.6 Complexidade

Assim como a Força Bruta, o Backtracking possui complexidade exponencial no pior caso, podendo chegar a $O(2^k)$, onde k representa o número de posições vazias do tabuleiro. Esse cenário ocorre quando as podas praticamente não eliminam estados da busca.

Na prática, entretanto, o desempenho é muito superior. Como diversas combinações inválidas são descartadas logo nas primeiras etapas da construção da solução, o número de estados efetivamente explorados é muito menor, reduzindo significativamente o tempo de execução.

O consumo de memória permanece $O(k)$, correspondente à profundidade máxima da recursão utilizada pelo algoritmo.

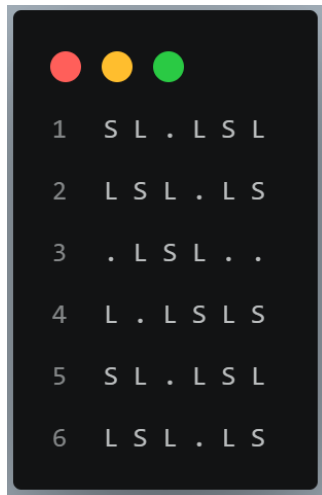
4. Exemplos de Execução

Para avaliar o funcionamento dos algoritmos implementados, foram utilizados três tabuleiros de diferentes níveis de dificuldade. Em todos os casos, tanto a Força Bruta quanto o Backtracking encontraram a mesma solução, porém com diferenças significativas na quantidade de estados explorados e no tempo de execução.

4.1 Tabuleiro 1 – Nível Fácil

O primeiro teste foi realizado utilizando um tabuleiro com poucas posições vazias, permitindo verificar se ambos os algoritmos produziam corretamente a solução esperada.

Arquivo de entrada



A Tabela 1 apresenta um resumo da execução dos dois algoritmos.

Métrica	Força Bruta	Backtracking
Tempo de execução	1 ms	0 ms
Estados explorados	9	10
Chamadas recursivas	0	9
Soluções encontradas	1	1
Podas realizadas	—	1

Como esse tabuleiro já estava quase completo, a diferença entre os algoritmos foi pequena, uma vez que poucas possibilidades precisavam ser analisadas.

A solução encontrada por ambos foi a mesma.

```
=====
FORÇA BRUTA
=====
Tempo de execucao: 0 ms
Estados explorados: 9
Chamadas recursivas: 0
Solucoes encontradas: 1

Solucao:
  0 1 2 3 4 5
0 | S L S L S L
1 | L S L S L S
2 | S L S L S L
3 | L S L S L S
4 | S L S L S L
5 | L S L S L S
Restricoes:
= (0,0) (1,1)
x (0,1) (0,2)
= (2,2) (3,3)
x (5,4) (5,5)

=====
BACKTRACKING
=====
Tempo de execucao: 0 ms
Estados explorados: 10
Chamadas recursivas: 9
Solucoes encontradas: 1
Quantidade de podas: 1

Solucao:
  0 1 2 3 4 5
0 | S L S L S L
1 | L S L S L S
2 | S L S L S L
3 | L S L S L S
4 | S L S L S L
5 | L S L S L S
Restricoes:
= (0,0) (1,1)
x (0,1) (0,2)
= (2,2) (3,3)
x (5,4) (5,5)
```

4.2 Tabuleiro 2 – Nível Médio

No segundo teste foi utilizado um tabuleiro com uma quantidade maior de posições vazias. Esse cenário permitiu observar de forma mais evidente o impacto das podas realizadas pelo algoritmo de Backtracking.

Arquivo de entrada

```
● ● ●
1  S L . L . .
2  L S L . . S
3  . L S L . .
4  L . L S . .
5  S L . L S .
6  L S L . . S
```

Métrica	Força Bruta	Backtracking
Tempo de execução	0 ms	0 ms
Estados explorados	5.286	22
Chamadas recursivas	0	16
Soluções encontradas	1	1
Podas realizadas	—	6

Observa-se que, enquanto a Força Bruta precisou explorar milhares de estados, o Backtracking

encontrou a solução analisando apenas algumas dezenas de possibilidades.

```
=====
FORÇA BRUTA
=====
Tempo de execucao: 0 ms
Estados explorados: 5286
Chamadas recursivas: 0
Solucoes encontradas: 1

Solucao:
  0 1 2 3 4 5
0 | S L S L S L
1 | L S L S L S
2 | S L S L S L
3 | L S L S L S
4 | S L S L S L
5 | L S L S L S
Restricoes:
= (0,0) (1,1)
x (0,1) (0,2)
= (2,2) (3,3)
x (5,4) (5,5)
= (1,1) (3,1)
```

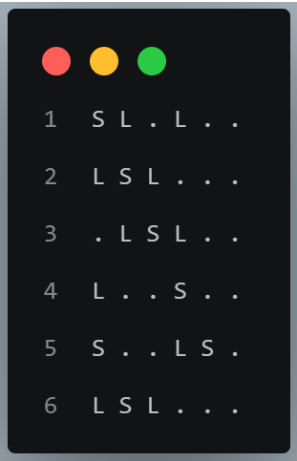
```
=====
BACKTRACKING
=====
Tempo de execucao: 0 ms
Estados explorados: 22
Chamadas recursivas: 16
Solucoes encontradas: 1
Quantidade de podas: 6

Solucao:
  0 1 2 3 4 5
0 | S L S L S L
1 | L S L S L S
2 | S L S L S L
3 | L S L S L S
4 | S L S L S L
5 | L S L S L S
Restricoes:
= (0,0) (1,1)
x (0,1) (0,2)
= (2,2) (3,3)
x (5,4) (5,5)
= (1,1) (3,1)
```

4.3 Tabuleiro 3 – Nível Difícil

Por fim, foi utilizado um tabuleiro com ainda mais posições vazias, aumentando significativamente o espaço de busca.

Arquivo de entrada



Métrica	Força Bruta	Backtracking
Tempo de execução	15 ms	0 ms
Estados explorados	76.203	28
Chamadas recursivas	0	20
Soluções encontradas	1	1
Podas realizadas	—	8

Nesse cenário, a diferença entre os algoritmos tornou-se ainda mais evidente. Enquanto a Força Bruta

precisou percorrer mais de setenta mil estados para encontrar a solução, o Backtracking explorou apenas 28 estados graças às podas aplicadas durante a construção da solução.

===== FORÇA BRUTA ===== Tempo de execucao: 17 ms Estados explorados: 76203 Chamadas recursivas: 0 Solucoes encontradas: 1 Solucao: 0 1 2 3 4 5 0 S L S L S L 1 L S L S S L 2 S L S L L S 3 L S L S L S 4 S L S L S L 5 L S L S L S Restricoes: = (0,0) (1,1) x (0,1) (0,2) = (2,2) (3,3) x (5,4) (5,5) = (1,1) (3,1) x (2,3) (3,3) = (4,0) (2,0)	===== BACKTRACKING ===== Tempo de execucao: 0 ms Estados explorados: 28 Chamadas recursivas: 20 Solucoes encontradas: 1 Quantidade de podas: 8 Solucao: 0 1 2 3 4 5 0 S L S L S L 1 L S L S S L 2 S L S L L S 3 L S L S L S 4 S L S L S L 5 L S L S L S Restricoes: = (0,0) (1,1) x (0,1) (0,2) = (2,2) (3,3) x (5,4) (5,5) = (1,1) (3,1) x (2,3) (3,3) = (4,0) (2,0)
---	---

Os resultados obtidos demonstram a eficiência do Backtracking na resolução de problemas de satisfação de restrições, reduzindo consideravelmente o espaço de busca sem comprometer a corretude da solução encontrada.

5. Análise de Complexidade

5.1 Espaço de Busca

Considere um tabuleiro de dimensão $N \times N$ contendo k posições vazias. Como cada uma dessas posições pode receber apenas dois símbolos (Sol ou Lua), o número máximo de configurações possíveis é dado por: $|\Omega| = 2^k$

Isso significa que o espaço de busca cresce de forma exponencial conforme aumenta a quantidade de células vazias, tornando inviável testar todas as combinações em instâncias maiores do problema.

5.2 Comparação entre os Algoritmos

Embora tanto a Força Bruta quanto o Backtracking sejam capazes de encontrar a solução correta, eles exploram o espaço de busca de maneiras bastante diferentes.

Aspecto	Força Bruta	Backtracking
Exploração dos estados	Analisa todas as combinações possíveis	Explora apenas estados que ainda podem gerar uma solução
Momento da validação	Apenas após o preenchimento completo do tabuleiro	Após cada nova atribuição
Utilização de podas	Não utiliza	Utiliza podas baseadas nas regras do jogo

Nos testes realizados, observou-se que a diferença entre os algoritmos aumenta conforme cresce o número de posições vazias do tabuleiro. Enquanto a Força Bruta percorre milhares de combinações até encontrar uma solução válida, o Backtracking elimina rapidamente os caminhos inviáveis, reduzindo significativamente a quantidade de estados explorados.

Por exemplo, no segundo tabuleiro a Força Bruta explorou **5.286 estados**, enquanto o Backtracking precisou analisar apenas **22 estados**. No terceiro tabuleiro, a diferença foi ainda mais expressiva: **76.203 estados** para a Força Bruta contra apenas **28 estados** para o Backtracking.

5.3 Impacto das Podas

O ganho de desempenho do Backtracking ocorre devido à aplicação de podas durante a construção da solução. Sempre que uma atribuição viola alguma das regras do quebra-cabeça, como a formação de três símbolos iguais consecutivos, o excesso de Sóis ou Luas em uma linha ou coluna, ou o descumprimento das restrições de igualdade e oposição, o algoritmo interrompe imediatamente a exploração daquele ramo da árvore de busca.

Essa estratégia evita que inúmeras configurações inválidas sejam analisadas até o final, reduzindo consideravelmente o espaço de busca e tornando a resolução do problema muito mais eficiente.

Vale destacar que os tempos de execução podem variar de acordo com o computador utilizado, a máquina virtual Java e as condições de processamento no momento da execução. Por esse motivo, a principal métrica considerada nesta análise foi a quantidade de estados explorados por cada algoritmo, pois ela representa de forma mais consistente a eficiência da estratégia adotada.

5.4 Conclusão

Os resultados obtidos demonstram claramente a diferença entre as duas abordagens estudadas. A Força Bruta garante que uma solução será encontrada, caso ela exista, porém precisa explorar um número muito elevado de possibilidades à medida que o problema cresce.

Já o Backtracking aproveita as restrições do próprio quebra-cabeça para eliminar estados inviáveis logo nas primeiras etapas da busca. Como consequência, encontra a mesma solução explorando uma quantidade muito menor de estados, evidenciando a eficiência dessa técnica para a resolução de Problemas de Satisfação de Restrições (CSPs), como o quebra-cabeça Tango.

Link para o código fonte: <https://github.com/RafaellaCristinaCSS/Tango>